Student Reflection Note: Cloud Computing Mini Project

Course: CS3081 (Cloud Computing)

Student Name: Prince Kumar Dubey Roll no: 23010103

Github: https://github.com/prince-dubey45/cloud-computing-mini-project

Section: B

---

1. what this project was About

In this project, my goal was to build a complete cloud system that takes normal views commands ("wake me up at 7 am" or "what is the weather today?") and turns them into clean JSON data.

Running a huge 3.8 billion parameter AI model on a server is way too slow and expensive. So I used knowledge distillation. I let a big teacher model (Phi3.5) generate labels for 3,000 commands, cleaned up all the errors, and used that data to train a tiny 70 million parameter model.

2. What I learned from this Project.

- Cloud Storage & Data Splitting

I learned how to set up SeaweedFS on kubernets to give me an S3-compatible storage bucket. Since running 3,000 commands on a single CPU takes hours, I learned how to use kubernetes Indexed Jobs to split the dataset evenly between two parallel workers using simple modulo math.

- Knowledge Distillation

I learned how Sequence distillation works in practice. Because my teacher model (Phi-3.5) and student model (Pythia-70m) use completely different token vocabularies, I could not match token probablities directly.

- Building clean Docker containers :-

I learned how to keep container images small. By using the official CPU-only PyTorch build and downloading model weights dynamically from our S3 bucket when the container starts up. I kept the Image size down to 1.14 GB well under 2GB limit.

~~Step~~

~~2. what~~

- CI/CD & load testing

I set up automated GitHub Actions to lint the code, run unit tests, scan the model against 95% validity gate and scan for leaked secrets. I also learned how to use locust to stress-test my API under load.

Challenges I Faced & How I Solved them

1. The Trailing Space Tokenizer Bug:

When I first ~~my~~ tested ~~my friend~~ student model, it scored 0%. Validity. It was either outputting blank strings or stopping immediately. When I inspected the token IDs, I realized my prompt ended with "JSON:" with a trailing space. In the tokenizer, "JSON:" plus a got merged into one token during training, but during live testing, the space was treated as a separate token.

2. Hardware Speed & training Stability:
the Reject Sheet mentioned that training on CPU takes 1 to 1.5 hours. On my Apple Silicon mac, training finished much faster because PyTorch utilized the unified CPU. However the default learning rate caused numbers to overflow into nan. I solved this by adding gradient clipping and making on padding tokens so loss only updated on real JSON tokens.

4. Key observations and Findings

- Data Cleaning Was ESsential: out of 3,000 raw teacher outputs, 109 had Problems. Exactly 2,891 clean commands remained.
- Small models can be surprisingly good:
- Untrained Pythia: 0% Validity, 0% accuracy
- Big Teacher Phi-3.5 (3.8 B Parameters): 100% Validity, 48% full frame accuracy.
- Distilled Student V1 (70M Parameters): 99% Validity, 43% full frame accuracy.

the tiny 70M model captured almost 90% of the giant teache
Rs accuracy while having only 1.8% of its parameter size!

Low memory Foot Print: The Service only used 52.23MiB of RAM at peak load, cold-started in just 3.35 seconds.
Load Test & Real Costs: Running 4 concurrent users for 90seconds gave 2.9 requests per second with a median latency of 1600ms.